

Python-Grundlagen für die Caesar-Verschlüsselung

Alle Bausteine, die du brauchst – in der Reihenfolge, in der man sie lernen sollte. Jedes Beispiel kannst du direkt in Python ausprobieren.

1. Ausgabe mit `print()` und Kommentare

`print()` zeigt etwas auf dem Bildschirm an. Alles hinter `#` ist ein Kommentar, den Python ignoriert. Kommentare sind Notizen für Menschen.

```
print("Hallo!")          # gibt Hallo! aus
print("Ergebnis:", 42)  # mehrere Dinge mit Komma trennen → Ergebnis: 42
```

2. Variablen

Eine Variable ist ein Name für einen Wert, wie eine beschriftete Schachtel. Mit `=` legst du etwas hinein.

```
schluessel = 3
text = "HALLO"
schluessel = schluessel + 1  # alten Wert nehmen, 1 dazu, wieder speichern → 4
```

Wichtig: `=` bedeutet „speichere“, nicht „ist gleich“. Variablennamen dürfen keine Leerzeichen und am besten keine Umlaute enthalten (`schluessel` statt `schlüssel`).

3. Datentypen

Für die Caesar-Verschlüsselung brauchst du vier Arten von Werten:

Typ	Beispiel	Wofür bei Caesar?
<code>str</code> (Text)	"HALLO"	der Text, das Alphabet
<code>int</code> (ganze Zahl)	3, -1	Schlüssel, Positionen
<code>bool</code> (Wahrheitswert)	True, False	Ergebnis von Prüfungen
<code>list</code> (Liste)	[0, 0, 0]	Buchstaben zählen (nur beim Knacken)

Text steht immer in Anführungszeichen. `"3"` ist Text, `3` ist eine Zahl. Das ist ein großer Unterschied!

4. Eingabe mit `input()` und `int()`

`input()` wartet, bis der Nutzer etwas tippt, und liefert es **immer als Text**. Für den Schlüssel brauchst du aber eine Zahl, deshalb wandelst du mit `int()` um.

```
text = input("Text eingeben: ")          # z. B. "Hallo"
schluessel = int(input("Schlüssel eingeben: "))  # "3" wird zur Zahl 3
```

Ohne `int()` funktioniert das Rechnen nicht: `"3" + 1` erzeugt einen Fehler.

5. Rechnen

```
7 + 3      # 10    Addition
7 - 3      # 4      Subtraktion
-3         # negative Zahl
28 % 26    # 2      Modulo: Rest beim Teilen
-1 % 26    # 25     Modulo funktioniert auch mit negativen Zahlen
```

Modulo `%` ist der Trick, mit dem das Alphabet „im Kreis läuft“. Wer ihn noch nicht kennt, kann stattdessen eine `while`-Schleife nutzen (siehe Abschnitt 10).

6. Strings (Texte) genauer

Ein String ist eine Kette von Zeichen. Jedes Zeichen hat eine Position, den **Index**, und Python **zählt ab 0**.

```
alphabet = "abcdefghijklmnopqrstuvwxyz"

alphabet[0]      # "a"    Zeichen an Position 0
alphabet[7]      # "h"
alphabet.index("h") # 7    an welcher Position steht "h"?
"h" in alphabet  # True   kommt "h" vor?
"!" in alphabet  # False
```

`alphabet[...]` und `alphabet.index(...)` sind Gegenstücke: Das eine macht aus einer Zahl einen Buchstaben, das andere aus einem Buchstaben eine Zahl. Genau das brauchst du für die Verschiebung.

Nützliche String-Methoden

```
"H".lower()      # "h"    in Kleinbuchstaben
"h".upper()      # "H"    in Großbuchstaben
"H".isupper()    # True   ist es ein Großbuchstabe?
"Mäuse".replace("ä", "ae") # "Maeuse" Zeichen ersetzen
```

Mehrere Methoden kann man hintereinander hängen: `text.replace("ä", "ae").replace("ö", "oe")`.

Strings zusammenkleben

Mit `+` hängst du Texte aneinander. So baust du ein Ergebnis Zeichen für Zeichen auf:

```
ergebnis = ""      # leerer Text zum Start
ergebnis = ergebnis + "K" # "K"
ergebnis = ergebnis + "D" # "KD"
```

7. Vergleiche

Vergleiche liefern `True` oder `False`.

Zeichen	Bedeutung	Beispiel
<code>==</code>	ist gleich	<code>wahl == "1"</code>
<code>!=</code>	ist ungleich	<code>zeichen != " "</code>
<code><, ></code>	kleiner, größer	<code>position < 0</code>
<code><=, >=</code>	kleiner/größer oder gleich	<code>position >= 26</code>
<code>in</code>	ist enthalten in	<code>"a" in alphabet</code>

Achtung: `=` speichert, `==` vergleicht. Das wird am Anfang oft verwechselt.

8. Entscheidungen mit if, elif, else

```
if zeichen in alphabet:
    print("Buchstabe – wird verschoben")
else:
    print("Kein Buchstabe – bleibt gleich")
```

`elif` („else if“) prüft weitere Fälle, z. B. für ein Menü:

```
if wahl == "1":
    print("Verschlüsseln")
elif wahl == "2":
    print("Entschlüsseln")
else:
    print("Ungültige Eingabe")
```

Mit `or` und `and` kombinierst du Bedingungen: `if wahl == "1" or wahl == "2":`

9. Einrückung – die wichtigste Regel in Python

Nach `if`, `else`, `for` und `while` kommt ein **Doppelpunkt**. Die Zeilen, die dazugehören, werden **eingerrückt** (üblich: 4 Leerzeichen). Die Einrückung entscheidet, was zusammengehört.

```
for zeichen in "ab":
    print("in der Schleife:", zeichen)    # läuft 2-mal
print("nach der Schleife")              # läuft 1-mal
```

Falsch eingerückter Code führt entweder zu einem `IndentationError` oder, schlimmer, zu einem Programm, das läuft, aber das Falsche tut.

10. Schleifen

for-Schleife über einen String

Geht jedes Zeichen einzeln durch. Das ist das Herzstück der Caesar-Verschlüsselung.

```
for zeichen in "HALLO":
    print(zeichen)          # H, A, L, L, O – je eine Zeile
```

for mit range()

Zählt Zahlen durch. Das brauchst du zum Knacken, um alle 26 Schlüssel auszuprobieren.

```
for zahl in range(26):  
    print(zahl)          # 0, 1, 2, ... 25 (die 26 ist nicht dabei!)
```

while-Schleife

Wiederholt, **solange** eine Bedingung wahr ist. Damit kannst du eine Zahl so lange verkleinern, bis sie in einen Bereich passt:

```
zahl = 60  
while zahl >= 26:  
    zahl = zahl - 26  
print(zahl)          # 8
```

Vorsicht: Wenn sich in der **while**-Schleife nichts ändert, läuft sie endlos. Abbrechen kannst du mit Strg + C.

Schleife in einer Schleife

Die innere Schleife läuft bei jedem Durchgang der äußeren komplett durch.

```
for zahl in range(3):  
    for zeichen in "ab":  
        print(zahl, zeichen)    # 0 a, 0 b, 1 a, 1 b, 2 a, 2 b
```

11. Listen (nur für das Knacken)

Eine Liste speichert mehrere Werte in einer Variablen. Wie bei Strings zählt der Index ab 0.

```
anzahl = [0] * 26          # Liste mit 26 Nullen  
anzahl[4] = anzahl[4] + 1  # Stelle 4 um 1 erhöhen  
max(anzahl)                # größter Wert in der Liste  
anzahl.index(max(anzahl))  # an welcher Stelle steht der größte Wert?
```

12. Typische Fehlermeldungen verstehen

Fehlermeldung	Was bedeutet das?	Typische Ursache
<code>IndexError: string index out of range</code>	Position gibt es nicht	<code>alphabet[28]</code> – Position über 25
<code>ValueError: invalid literal for int()</code>	Text ist keine Zahl	Bei der Schlüsseleingabe Buchstaben getippt
<code>ValueError: substring not found</code>	Zeichen nicht im String	<code>alphabet.index("H")</code> – Großbuchstabe nicht umgewandelt
<code>TypeError: can only concatenate str</code>	Text und Zahl vermischt	<code>int()</code> bei der Eingabe vergessen
<code>IndentationError</code>	Einrückung stimmt nicht	Einrückung nach <code>for/if</code> vergessen oder Tabs und Leerzeichen gemischt
<code>SyntaxError</code>	Python versteht die Zeile nicht	Doppelpunkt oder Anführungszeichen vergessen

Empfohlener Lernweg

1. Abschnitte 1 bis 4: Ein- und Ausgabe, Variablen, Datentypen
2. Abschnitte 6 und 10: Strings und die for-Schleife
3. Abschnitte 7 bis 9: Vergleiche, if und Einrückung
4. Abschnitt 5 und die while-Schleife: Zahlen in einen Bereich bringen
5. Abschnitt 11 und range(): erst für das Knacken ohne Schlüssel